A Novel Method for Upscaling of Optimal Topology using Deep Neural Networks.

Anirudh KANTHAMRAJU[1] and Joseph MORLIER[2]

Institut Supérieur De L'aéronautique Et De L'espace (ISAE Supaero) – Toulouse 31400 France.

Abstract

This paper proposes a novel methodology to predict optimal topologies at a higher mesh resolution from an input from a lower mesh resolution using deep neural networks. The experiments performed prove the capability of a *'Generative Adversarial Network'* and a *'Perceptual Loss Function'* to predict near optimal topology. The experiments performed proved the capability of the Network Architecture to bridge the gap between the assumptions in the current state of the art and the reality, i.e., mapping the features of *true low-resolution topology* to that of a *high-resolution topology* instead of mapping features from downscaled high resolution to their corresponding high-resolution output. The study uses a modified version of the network architecture which has already proven capability in upscaling low-resolution real-world images. Furthermore, the study also explores the application of a perceptual loss function to not penalize the network for predicting the outputs that are off by a pixel or two. The dataset used for the experiments were generated by using the Multi-material Topology Optimization code which is an extension of O.Sigmund's 88-Line MATLAB code. The dataset consists of 250 ordered pairs of low- and high-resolution topologies.

I. Introduction.

The problem of topology optimization aims to determine the optimal material(s) distribution subjected to a particular loading and boundary conditions while also adhering to the constraints of volume fractions and non-design domains within the allowable design space. In the context of design of structures, the methodology has been used in the design of Automotive, (R. J. Yang, 1995) and Aerospace (Lars Krog) structures. With the landmark paper (Sigmund) introduced, the now widely used mathematical framework for topology optimization called *'Solid Isotropic Material with Penalization'.* The method uses a minimum compliance as the objective function and penalizes the intermediate densities to obtain the optimal topology for a given set of loading and boundary conditions subject to a particular volume fraction. At the heart of the algorithm, the methodology uses a finite element subroutine along with a filtering algorithm to update the optimal material distribution. However, the method is iterative, and the time taken for the convergence depends highly on the finite element discretization of the design domain (ref Fig 1). This is a problem when higher mesh sizes are used because, the convergence time almost doubles when the mesh size is increased by a factor of two. Furthermore, in the case of multilateral distribution, the time for convergence is directly proportional to the number of materials used.

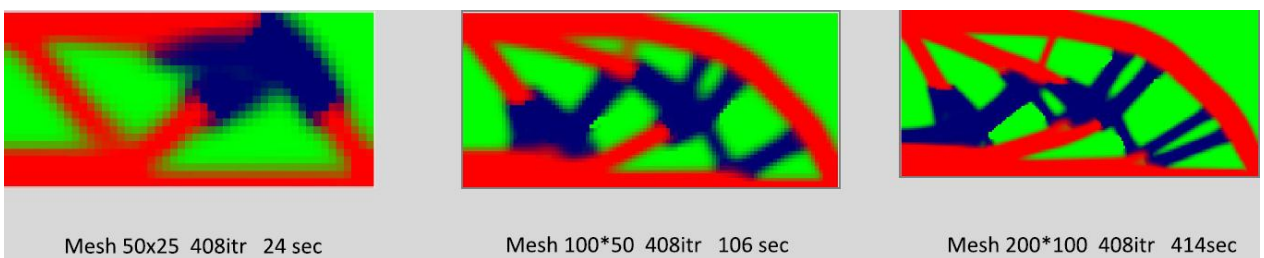


Figure 1 Convergence times for various mesh sizes

Generative algorithms are being used these days to solve the problem of topology optimization. (Orban, 2020). At the heart of these algorithms are data driven neural networks that can predict optimal topologies. A similar approach can be envisaged to predict a high-resolution topology from a low-resolution input. In the paper published by (Christian Ledig, 2017) a GAN architecture was proposed with the redefinition of the loss function to solve the problem of image super resolution. The loss function consisted of a perceptual loss function along with adversarial loss. the combination of the two enabled the network to predict features in higher resolution images that are very similar to the real high-resolution images. This was done by using CNNs which are capable of feature mapping. The ill posed nature of increasing the resolution of a low-resolution image is a particularly difficult at a upscaling factor of even 2x as the details of branched trusses is completely absent in the low resolution image as can be seen in figure 1. Therefore, in this work, we propose a Super-resolution GAN with skip connections and deviate from the traditional Mean Squared Pixelwise Loss to perceptual loss in order to achieve the high-resolution output.

II. Topology Optimization.

With his landmark paper in 1989, M.P Bendose, defined the topology optimization problem as an optimization problem (Bendose, 1989). Ever since then, the method of SIMP (Solid Isotropic Material with Penalization) method has been widely used in 2D and 3D applications (M.P.Bendose, 2002). The inclusion of a Finite Element based approach and penalization scheme to penalize the intermediate relative densities has enabled the algorithm to generate optimal structural layouts for any static loading condition. Mathematically a binary topology optimization can be defined as follows:

$$\min_{x} : \qquad c(x) = U^T K U = \sum_{e=1}^{N} (x_e)^p \; u_e k_o u_e$$

$$subjected\ to: \qquad \frac{V(x)}{V_0} = f,$$

$$\mathrm{KU} = \mathrm{F},$$

$$0 < x_{min} \leq x \leq 1$$

Where *c(x)* represents the cost function. F and u represent the global force and displacement matrices, respectively. x denotes the vector of design variables. u_e and k_o represent the elemental displacement and stiffness matrix (Bendose, 1989). In the case of multi-material topology optimization the penalization can be mathematically represented as follows (MOHSENI, 2013). Where the E1,2 represent the Young's Moduli of the phases and ρ is the penalization factor.

$$E(\rho) = \rho^p E_1 + (1\text{-}\rho^p) E_2$$

One major hurdle would be to generate enough data to train the neural network. And this is computationally very expensive. Furthermore, as discussed above, the mesh size would play a key role in the time taken to solve a particular loading condition. This would mean that a basic code with best possible efficiency must first be developed or chosen from the literature to run the test cases and generate the database for training. Furthermore, the code that has been chosen needs to be able to generate images that will be easier to process while training the Neural Network. A few of the key features that the code needs to have are the discrete separation of different phases, the material distribution in a structured mesh, a proper connectivity of the interfaces. The code published by (Emily D. Sanders) has the capability to solve the afore mentioned issues. However, the code implements the Poly-Mesh algorithms of MATLAB and thus the primary requirement of having a structured mesh in the output is not satisfied. This is an important feature that is needed in the code because, if the material distribution is in a structured mesh and each element of a mesh corresponds to each individual pixel, the setup of the initial layer also called the input layer would be significantly simpler. This is because the input layer would be a matrix whose dimension will have to be specified.

Therefore, the code published by (MOHSENI, 2013) is being used. The code delivers the outputs in the form of a structured mesh. The code implements a structured mesh and uses a bilinear approximation to generate the bitmap output. The code is an extension of the widely used 88-line MATLAB code published by (Sigmund).

III Description of the Data Set.

The Data set that is used in this study is a self-generated data set that was created by calling the *'Multitop'* function written my (MOHSENI, 2013). The description of the entire set is described in table 1.

Boundary Conditions	Cantilever, MBB
Mesh Size Low Resolution	40x20 , 100x50
Mesh Size High Resolution	80x40 , 200x100
Material Young's Moduli	[3, 2, 1]
Void Young's Modulus	1 e^{-9}
Void Volume Fraction range.	0.2 – 0.5: increment 0.01

Table 1 Description of the Data Set

To obtain a wide range of data points, the materials were distributed in the following ratios .

2 Material		
1:01	2:01	3:01
1:02	2:03	3:02
1:03		

1 Material
1:1

3 Materials		
1:01:01	**2:01:01**	**3:01:01**
1:01:02	**2:01:02**	**3:01:02**
1:01:03	**2:01:03**	**3:01:03**
1:02:01	**2:02:01**	**3:02:01**
1:02:02	**2:02:03**	**3:02:02**
1:02:03	**2:03:01**	**3:02:03**
1:03:01	**2:03:02**	**3:03:01**
1:03:02	**2:03:03**	**2:03:02**
1:03:03		

From all the possible combinations of volume fractions and material distribution ratios, a sample of 250 equidistant data points were selected. From these selections, the Multitop function was called to generate the complete data set.

IV Neural Network Basics.

In the broadest sense deep learning is a subset of Machine learning. Deep learning involves draining of neural network with a huge compilation data so that, after the training process is completed, the network has the capability of classifying, predicting and making certain decisions based on the data it was trained on. With the improvements in computational power and the developments in the field of parallel processing, these networks are capable of detecting features or patterns from the input data set and develop an *intuition* on the correlations between the input data and their corresponding features. Most notably, deep learning algorithms have been applied to the fields of text predictions, image recognition, driverless cars, weather forecasts oil refineries etc. In the context of this report only the most important components will be discussed.

One of the most noticeable examples of deep learning that most of us use is in the form of auto correct and text predictions on our phones. The fundamental difference between conventional coding and coding neural networks is that in a conventional code, the programmer needs to explicitly program response of the code to a particular input. However, no such programming is necessary for a neural network to work. Therefore neural networks have gained popularity in the recent years in the field involving optimization. The only drawback with neural networks is that it requires a large amount of data to be trained and accurately deliver results.

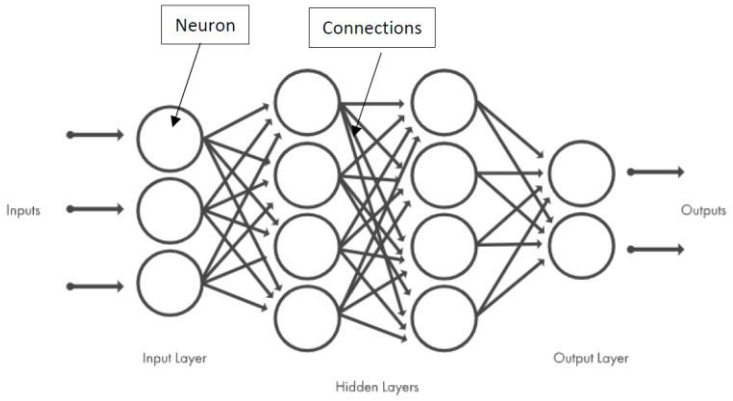


Figure 2 Visualization of a Generic Neural Network

The fundamental building block of any neural network is a neuron. Mathematically, each neuron is a mathematical function that returns a binary or a numerical output depending on the inputs and the activation function. The input of each neuron is a weighted average of the outputs of the preceding layer. The output of a neuron is determined by a mathematical function called an 'Activation Function' that either outputs a binary output or a particular value proportional to the input it receives. For example, the most common Activation function is the ReLu function that is a non-binary activation function. With the idea of keeping the report concise, this paper is only going to present the general neural networks architectures that are going to be used in the study while concisely explaining any terms which are necessary in the middle.

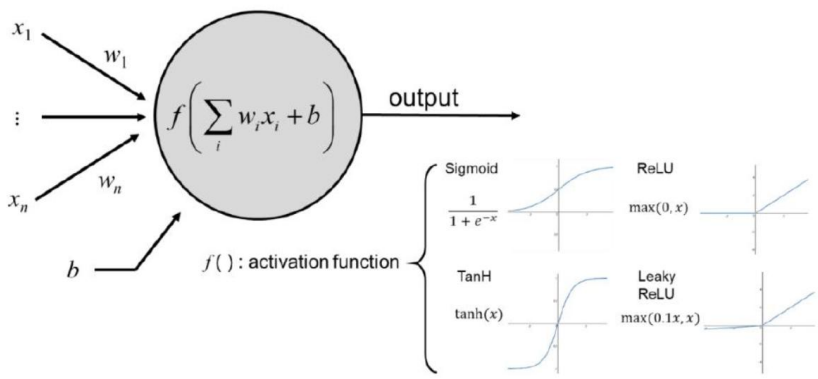


Figure 3 Symbolic Representation of a Neuron

Convolutional Neural Networks.

The CNN is the most common neural network architecture which has a widespread application in the fields of signal processing and image analysis. CNNs are fully interconnected and it's this interconnectivity between all the neurons of the layers that enable the nets to perform the above-mentioned tasks. However, this full interconnectivity results in the networks being over fit to the training data. One of the methods that is used to prevent the overfitting the data is the *'skip connections'* which will be explained in the later part of this paper. In a conventional image recognition algorithm, the features recognized by the algorithm are determined by a predefined filter (filter is a kernel of a specified dimension with predefined value, that is iteratively run over the entire image matrix to pick-out certain features). However, the CNN has the capability of updating the values of the filters as the training of the network progresses thus directly eliminating the need to specify the values of the kernel of the filter. A CNN can be described as a network in which a neuron is connected to only a subset of the previous neurons. It is due to this sparce connectivity that the network possesses the capability of accurately picking out the features of the images. The most generic implementation of a CNN is illustrated in the figure below.

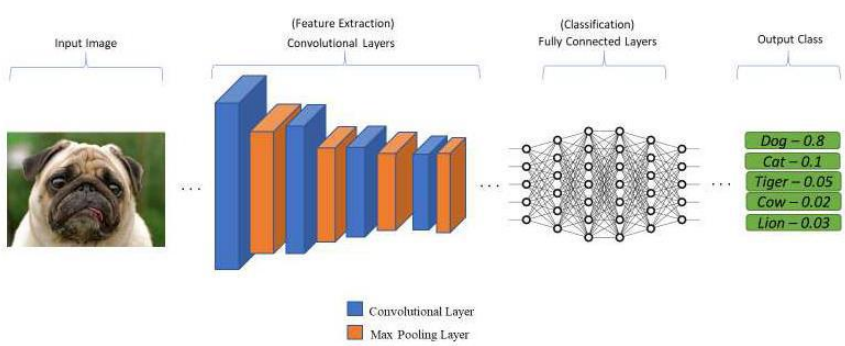


Figure 4 A generic representation of a CNN

A CNN is the fundamental building block of the present study. If looked at from a broader perspective the task of the network is to map the localized features of the low-resolution image to the

corresponding high-resolution image. In the generation of the dataset, the radius of filter of '3' was chosen. This number essentially links the mesh size to the model size. Therefore, the present papers use the filter radius of 3 as a starting point for the study. (Liang Xue) et al have established some correlation between the filter radius and the kernel sizes for the neural network. However, the implementation of those correlations in the context of this study, is difficult in regard to the sheer number of trainings that need to be performed. A more nuanced way to implement these correlations are explained in the *'Future Works'* section for the future research.

Generative Adversarial Networks (GANs).

In 2014 (Goodfellow) proposed a new neural network framework that estimates the generative models while simultaneously training two networks that are adversarial. The first network is the *'generator'* that is, and as the name suggests, can generate new approximations from an input. The second is the *'discriminator'* that can estimate the probability of a sample coming from either the training data or as an output from the generator. This class of neural network architecture enabled training in the form of unsupervised learning. The key idea of the GANs is based on an *'indirect'* training through the discriminator and the objective of the generator is to generate the sample such that the discriminator is not able to tell the difference between the generated data and the ground truth. GANs have been widely used in the various industries and one of the most significant application of GANs is in image upscaling in Video Games. Mathematically speaking the goal of the GAN is to minimize the loss between the generator and the discriminator. Here, p_{data}(y) and p(z) are the probability density functions of real data y and latent variable 'z'.

$$L^{GAN} = E_{y \sim p_{data}(\mathbf{y})}[\log(D(\mathbf{y})) + E_{\mathbf{z} \sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z}))]$$

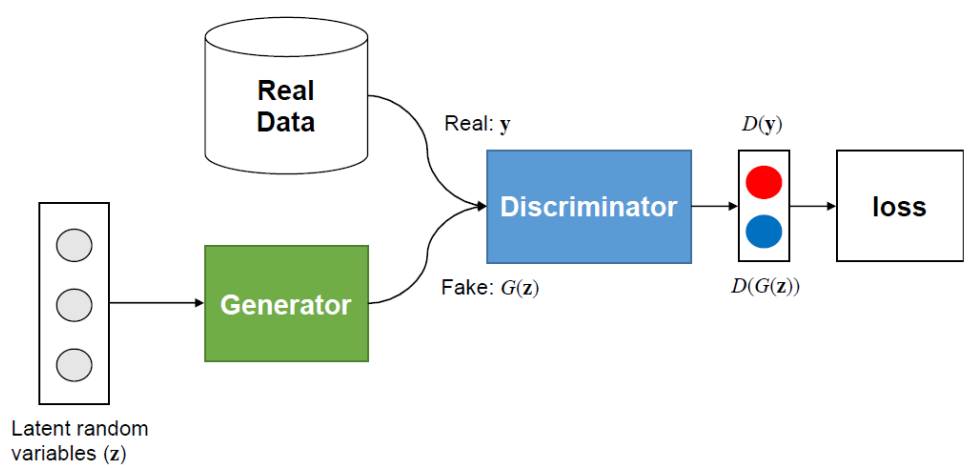


Figure 5 A simple GAN Architecture

V Neural Network Architecture of the study

Requirements of the Neural Network – Global Objectives

In the field of Image Processing and Video Game Industry, the problem of *'Single Image Super Resolution (SISR)',* is a widely studied topic since neural networks can produce high resolution, realistic looking images for a fraction of the computation cost needed to generate them. Many Neural Network architectures have been proposed to efficiently tackle the problem. However, the context of topology optimization, to the best knowledge of the authors, the problem of *'Multi-material Topology Optimization'* has not been attempted using the image upscaling techniques. Furthermore, in the space literature that was available, problem of topology optimization was ill posed.

(Nicolas Napier) proposed a fully connected, forward feed ANN architecture with a single hidden layer of 15 nodes. The data set used in the study were 20 images that were generated using (Sigmund) MATLAB code. The high resolution sample obtained by the MATLAB code were then downscaled to produce the low resolution input. The objective of the research was to optimize the edges of a lower resolution topology to improve the resolution.

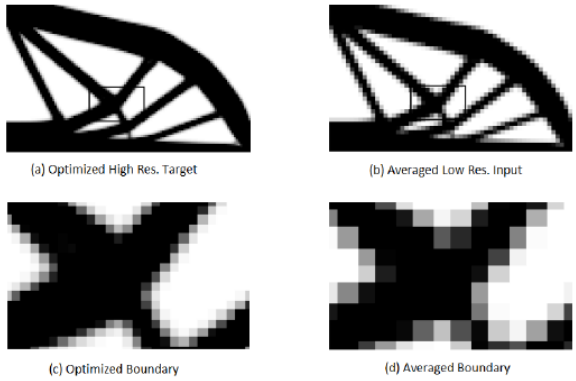


Figure 6 Optimized Structure Before and after averaging.

However, after analyzing the ordered pairs of low- and high-resolution data points, even for a single material case, this approximation is not valid since the optimized structure evolves with the increase in the mesh size. And even for upscaling factor as small as 2x the optimized structure has more features that the lower resolution input does not. Thus, classifying the problem of *'true'* topology upscaling as upscaling the edges of the edges of the optimized structure is definitely ill posed.

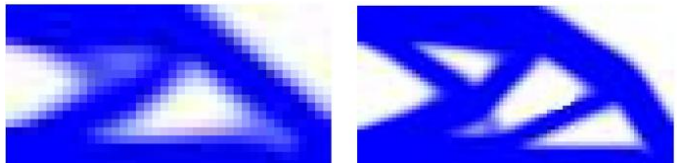


Figure 7 Evolution of Optimal topology for 2x upscaling of mesh size

Furthermore, even if the *true* high-resolution image was downscaled using pixelwise algorithms, the resultant low-resolution image will have *blurred artifacts* from downscaling, which any CNN will be able to identify as a feature and then map the blurred artifacts into the prediction. This will be a problem when the network will be subjected to an input of an unseen *true* low-resolution image because, the blurred artifacts will be definitely absent in the input.

The choice of the loss function also plays a very important role in the training of the neural networks. In a very crude form, the objective of a loss function is to guide the network onto the right direction at every instance of the training. Classically for the problem of the image super resolution, a classic loss function for image analysis i.e. *'Mean Squared Error (MSE)'* was used. Mathematically MSE can be described as follows.

$$Loss = \frac{1}{n}\sum_{N=1}^{n} \|H_N(L,\ W_l,\ B_l) - H_N^{\text{real}}\|^2,$$

Where n is the number of training samples selected for each iteration. The value of l is 1, 2, and 3 corresponding to three convolution operations. H is the high-resolution configuration of L reconstructed by SRCNN, and H_{real} is the high-resolution training sample matched with L. The random gradient descent method is used to minimize the loss value during standard backpropagation (Liang Xue).

The results obtained from the state of the art according to (Christian Ledig, 2017) by employing MSE are perceptually unsatisfying. In the application for topology optimization, the objective of the generator when the training commences is to produce optimal topologies that are close *'approximations'* of the true optimal topology. However, MSE will result in a higher loss even if the optimal topology is off by a few pixels. Thus a relatively nuanced loss function needs to be defined. Such as the perceptual loss function as described by (Christian Ledig, 2017)

Neural Network Architecture – Description

For the reasons as described above, the goal of the Neural Net should be to predict the *ideal optimal high-resolution topology* for the *true low-resolution* input. The solution to such an objective is described in the paper by (Christian Ledig, 2017). In the paper they propose a *'SRGAN – Super Resolution Generative Adversarial Network'* that is capable of upscaling the input image to 4x with the definition of a custom loss function that they called *'Perceptual Loss function'*. The network architecture used in this study is the exact same as the ones they used however with a slight difference in both the network architecture as well as the characteristics of the training and validation data.

The generator consists of 16 residual blocks (B=16) with identical layout (not visualized in figure 8). In each block we have two convolutional layers each followed by a Batch Normalization layer and ParametricReLU as the activation function. The image resolution is increased by 2x by using an Up-sampling Layer.

The discriminator consists of 6 convolution layers with an increasing number of 3x3 filters increasing from 64 up to 256. The resulting feature maps are then followed by two dense layers and a final sigmoid activation to obtain the probability for sample classification.

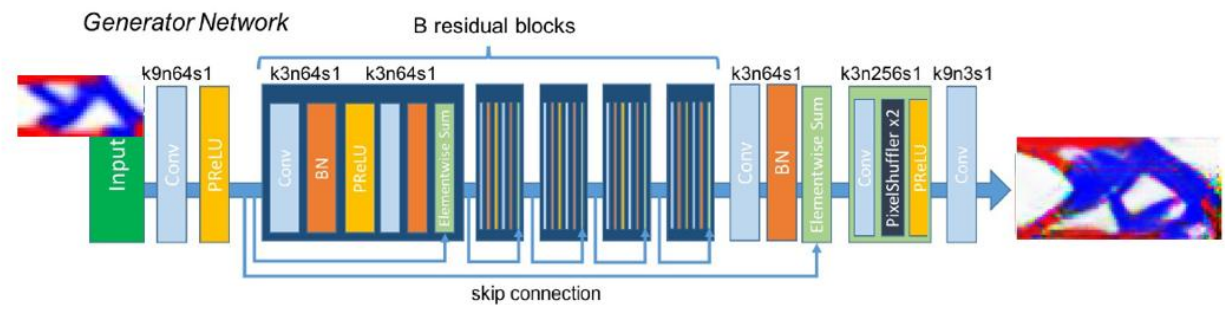


Figure 8 Generator Architecture

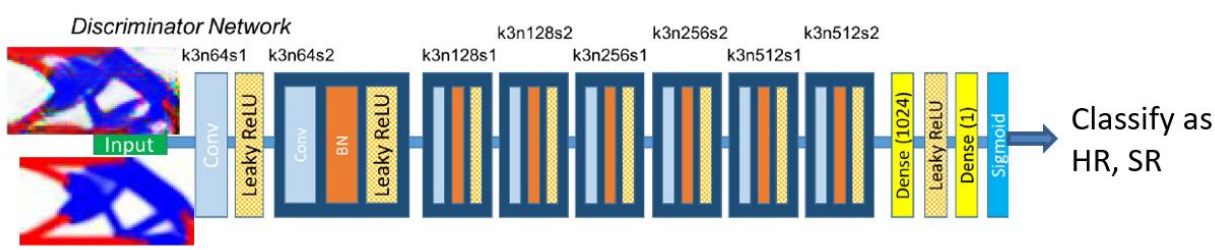


Figure 9 Discriminator Architecture.

Loss Function

The loss function in this study can be broken up into 2 parts. The *'Content Loss'* and the *'Adversarial Loss'*. The content loss or the VGG loss is based on the ReLu activation layers of the nineteen layered , pre trained VGG Network. Mathematically VGG Loss can be described as follows.

$$l^{SR}_{VGG/i.j} = \frac{1}{W_{i,j}H_{i,j}} \sum_{x=1}^{W_{i,j}} \sum_{y=1}^{H_{i,j}} (\phi_{i,j}(I^{HR})_{x,y} - \phi_{i,j}(G_{\theta_G}(I^{LR}))_{x,y})^2$$

Where, $\emptyset_{i,j}$ represents the feature map obtained by the jth convolution before the ith maxpooling layer in the trained VGG19 network. Then the VGG loss can be defined as the Euclidean distance between the feature representations of a reconstructed image $G_{\theta G}(I^{LR})$ and the reference image I^{HR}. $W_{i,j}$ and $H_{i,j}$ represent the respective dimensions of the feature maps in the VGG network.

The *'Adversarial Loss',* as described previously drives the generator in the direction where it can reach a point where it's outputs can fool the Discriminator. This is done to favor the solutions that lie on the manifold of the true optimal topologies. The generative loss can be described as

$$l_{Gen}^{SR} = \sum_{n=1}^{N} -\log D_{\theta_D}(G_{\theta_G}(I^{LR}))$$

Here $D_{\theta D}$ $(G_{\theta G}(I^{LR}))$ is the probability that the reconstructed image $G_{\theta G}(I^{LR})$is a true optimal topology.

VI Experiments

The objective of this study was to evaluate the Network Architecture to predict the optimal topology in its existing state. So, as an initial proof of concept three experiments were performed. The descriptions of which can be found below.

Experiment	Boundary Conditions	Mesh Size input	Number of Epochs	Time of computation	Number of data Points
Exp-1	Cantilever	40x20	5000	22 Hrs	251
Exp-2	Cantilever	100x50	5000	24 Hrs	251
Exp-3	Cantilever + MBB	40x20	6000	30 Hrs	502

Table 2 Description of the Experiments

The data set was split into two independent sets. Testing data that consists of 80% of the data and the validation containing the remaining 20% of the data. The environment that was used for the study is as follows.

- Python – 3.8.8
- TensorFlow – 2.4.1
- Training was performed on Nvidia GPU

VII Results and Conclusions.

In all the three experiments, the model was able to successfully map the features of the low-resolution image to that of the high resolution output. This is despite the absence of the artifacts that would have emerged from downscaling the high-resolution outputs using pixelwise algorithms. Thus it can be concluded with an absolute certainty that the existing state of art from image processing domain can be directly be applied with absolutely no modifications.

If the predictions of a single, dual, and triple material cases were looked at independently, the predictions of the single material case have been the best followed by the dual and triple material case. And if we evaluate the outputs of each of the experiments, the outputs of the first experiment were by far the best. This can be explained by the fact that it is the simplest of the three experiments with significantly smaller mesh size and only approximates the topologies of a single boundary condition. This fact is also re-enforced by the fact that the entire training of 5000 epochs was completed in around 22 hours. The generator was able to generate the optimal topologies in 35 seconds. This is slightly higher than its iterative counterpart that can generate the outputs in 25 seconds.

The Experiment-2 has quite a lot of noise in them despite training for almost 5000 epochs over 24 hours. However, the network at the end of 5000 epochs is quite capable of approximating the shape of the optimal topology quite accurately. However, in the prediction of optimal structures involving two or three materials, the network struggles in the resolution of the material boundary. However, even in this case the network can estimate the edges of the structure quite well. This could be resulting from the complexity of mapping into a significantly bigger output in comparison to Experiment 1. There was a significant increase in the number of weights and biases when the input and output resolutions were increased in size. However, the prediction time is 64 seconds in comparison to almost 100 seconds for the iterative counterpart. It can be said with an absolute certainty that the computation time will increase as the mesh size increases. However, the increase will not be as severe as in the case of the conventional algorithms.

The Experiment -3 was the most complex experiment that was performed. The network was trained on data set that was double that size and which consisted of two boundary conditions. Despite its smaller mesh size, in comparison to Experiment-2 the feature extractor was not able to correctly extract the features from both the boundary conditions. The trends in the quality of predictions in single, dual, and triple material case is followed even in this case. The training was carried out for 6000 epochs over 30 hours. This experiments necessitates the need for a better feature extractor if the existing architecture were to be used.

NOTE : The outputs of the neural networks will be presented in the appendix of this paper.

VIII Scope of the Future Work.

The research tries to solve the problem of prediction of optimal topology by using implementations from existing state of the art from the field of image processing. Despite its capabilities, the network architecture is still in its infancy and there is a lot of scope in improving the model. The first mode is by implementing a compliance predictor either from FEM or through Deep Neural Networks, that will be able to predict the compliance of the predicted topology during the training process. The error between the predicted compliance and the ground truth can be used as another addition to the loss function during the training process.

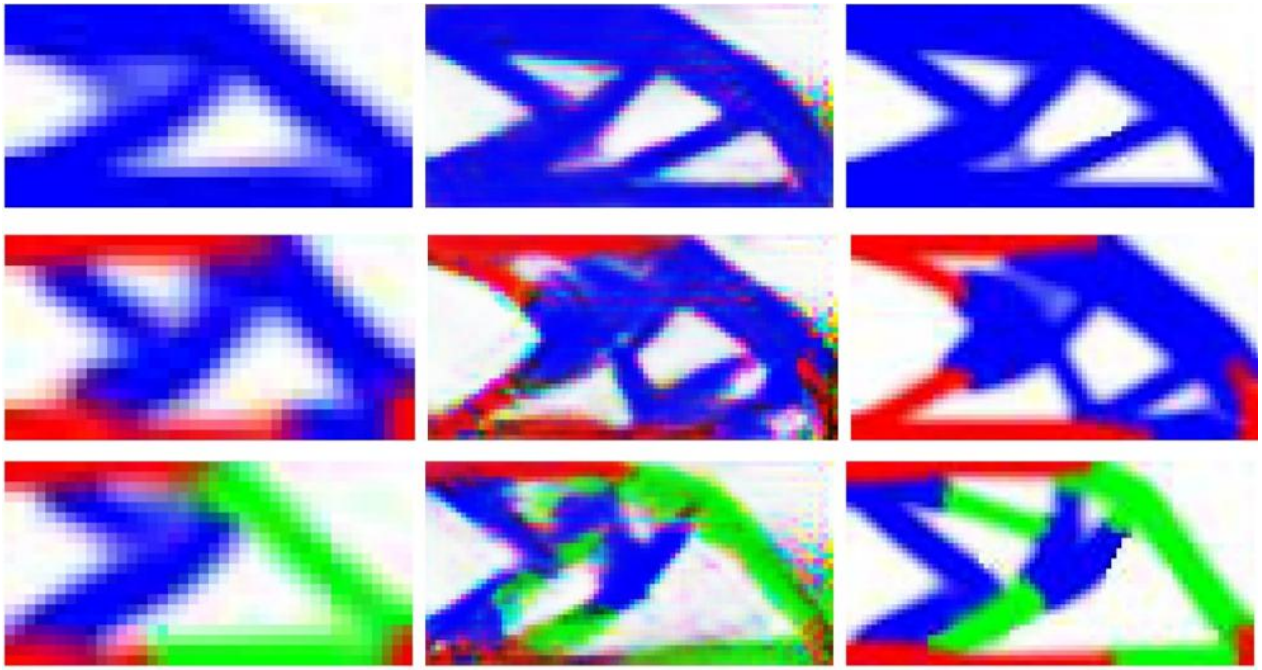
Right now, the features extracted by the network are still unknown. The developments in the field of image processing have led to the creation of diagnostic tools such as **'Weights and Biases'** which can visualize the feature extractor's outputs. These tools can be used to fine tune the models feature extractor which is at the core of the study that is being performed. Experiments with varying kernel size can be performed to evaluate the networks performance. The key to solving the problem of handling multiple boundary conditions lies within the problems of Experiment-3.

Since Numpy can support multiple input channels, the input image can also contain stress and displacement fields that can be used in the training of the network. Furthermore, if the network were to be implemented in a commercial engineering software, the stress and displacement fields would be of a great interest to the end user.

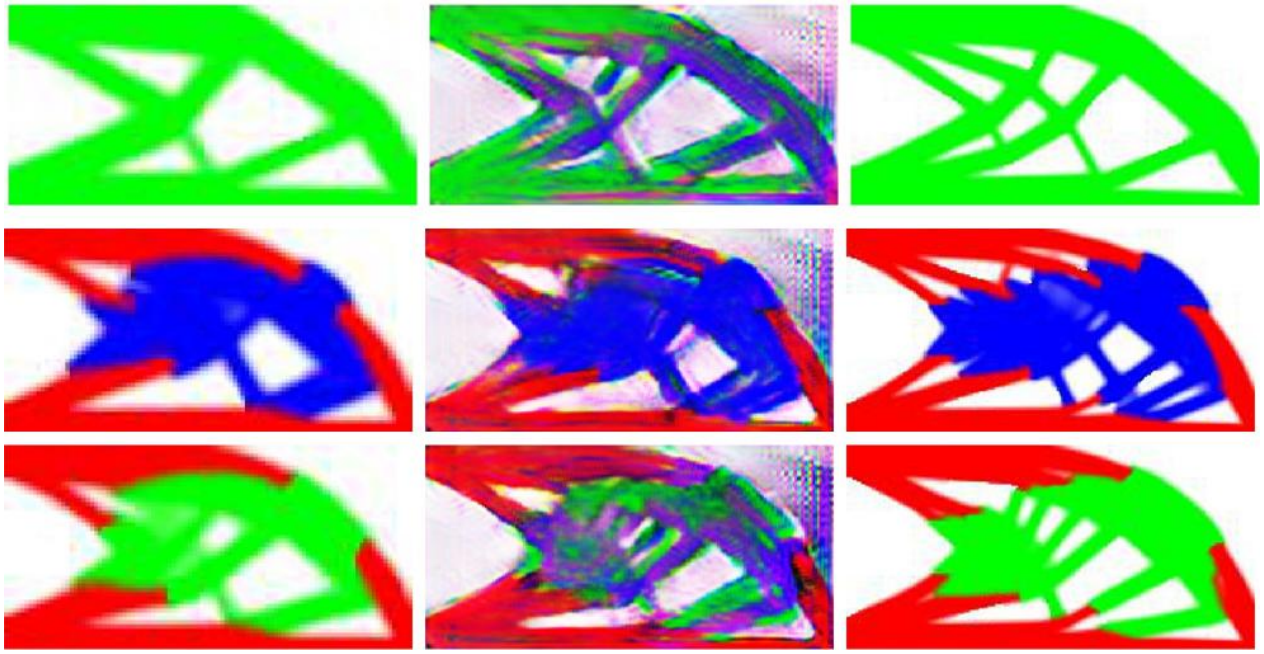
IX Appendix.

Experiment-1

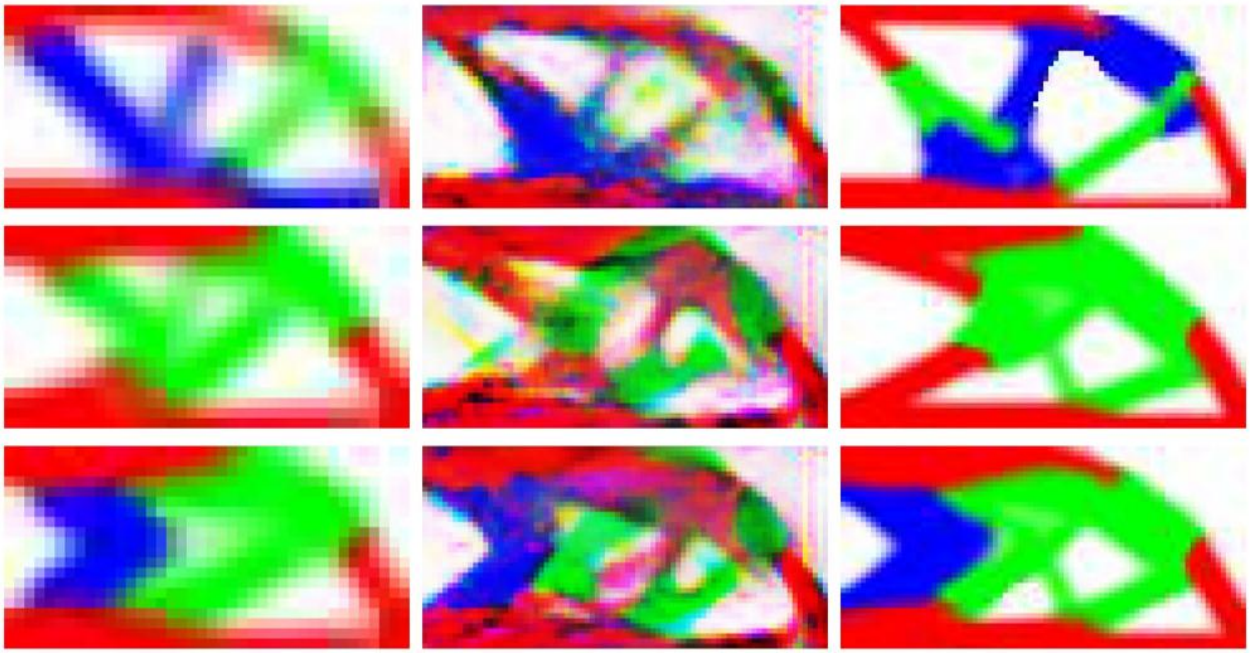
Low-Resolution Input **Generator Output** **True High-Resolution Output**



Experiment-2



Experiment-3



X References.

Bendose, M. (1989). Optimal Shape Design as a Material Distribution problem. *Structural and Multi-Disciplinary optimisation*.

Christian Ledig, L. T. (2017). Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial.

Emily D. Sanders, A. P. (n.d.). *Where the E1,2 represent the Young's Moduli of the phases and ρ is the penalization factor. .*

Evangelos Tyflopoulos, M. S. (2018). State of the art of generative design and topology optimization and potential . *Research Gate*.

Goodfellow, I. (n.d.). generative adverserial Networks.

Lars Krog, A. T. (n.d.). Application of Topology, Sizing and Shape Optimization Methods to Optimal Design of Aircraft Components.

Liang Xue, J. L. (n.d.). An efficient and high-resolution topology optimization method based on convolutional neural networks.

M.P.Bendose, O. S. (2002). *Topology Optimisation. Theory, Methods and Applications.*

MOHSENI, R. T. (2013). ALTERNATING ACTIVE-PHASE ALGORITHM FOR MULTIMATERIAL TOPOLOGY OPTIMISATION PROBLEMS 115-LINE MATLAB IMPLIMENTATION.

Nicholas Napier, S.-A. S. (n.d.). An Artificial Neural Network Approach to Generating High-Resolution Designs from Low-Resolution Input in Topology Optimization.

Nicolas Napier, S. A. (n.d.). An Artificial Neural Network Approach to Generating High-Resolution Designs from Low-Resolution Input in Topology Optimization .

Orban, R. (2020). *Topology Optimization is not Generative Design.* Retrieved from autodesk.com: https://www.autodesk.com/products/fusion-360/blog/topology-optimization-is-not-generative-design/

R. J. Yang, A. I. (1995). Automotive applications of topology optimization.

Rozvany, G. (n.d.). Aims, scope, methods, history and unified terminology of Computer Aided Topology Optimisation in Structural Mechanics .

Sigmund, O. (n.d.). A 99 Line Topology Optimization Code Written in MATLAB. Structural and Multidisciplinary Optimization.